PLATAFORMA · VERSIÓN 1.0 · MAYO 2026

Documentación técnica de plataforma.

Arquitectura, stack tecnológico, modelo de datos, flujos de negocio, controles de seguridad y cumplimiento normativo de la tienda online y panel de gestión de Madcry Studio (madcry.com).

00 Datos del documento

Versión, alcance y responsables

DOCUMENTO	Documentación técnica de plataforma — madcry.com
VERSIÓN	1.0 · Mayo 2026
PROPIETARIO DEL PRODUCTO	Susana Madriz · Madcry Studio
IMPLEMENTACIÓN Y MANTENIMIENTO	Tane Solutions (consultora digital)
REPOSITORIO	`github.com/JohnSM7/tienda-chachimad` (privado)
ENTORNO PRODUCTIVO	https://madcry.com — Firebase Hosting (CDN global)
ALCANCE	Sitio público de catálogo, e-commerce, formulario de contacto, blog y panel de administración.
AUDIENCIA OBJETIVO	Auditoría técnica, due diligence, evaluación legal, soporte interno.

↗ Tabla de contenidos

Estructura del documento

01 **Resumen ejecutivo** Qué es la plataforma y cómo está construida 03

02 **Stack tecnológico** Tecnologías y proveedores en uso 04

03 **Arquitectura** Frontend, backend, infraestructura 05

04 **Modelo de datos** Esquema de base de datos y relaciones 06

05 **Flujos críticos** Compra, contacto, publicación de obra 07

06 **Seguridad — capas y controles** Defensa en profundidad 09

07 **Gestión de secretos** Credenciales, rotación, exposición 11

08 **SEO, rendimiento y accesibilidad** Indexación, Core Web Vitals, WCAG 12

09 **Cumplimiento legal** RGPD, LSSI-CE, datos personales 13

10 **Operación, backups y continuidad** Despliegue, recuperación, monitorización 14

01 Resumen ejecutivo

Visión general en una página

Madcry Studio es la galería online y tienda directa de la artista **Susana Madriz**. La plataforma permite mostrar el catálogo de obra, gestionar la venta de piezas únicas (1/1) con cobro a tarjeta, recoger comisiones a medida a través de un formulario de contacto y administrar todo el contenido desde un panel privado.

El sitio se construye con **Astro 5** en modo estático (SSG): cada despliegue genera HTML pre-renderizado servido desde una CDN global. La interactividad puntual (panel admin, botón de compra) se hidrata con **Vue 3**. El backend transaccional vive en **Supabase** (PostgreSQL gestionado, autenticación, almacenamiento de imágenes y funciones serverless), mientras que el cobro lo realiza **Stripe** en modo Checkout, sin que datos de tarjeta lleguen nunca al servidor de Madcry.

CARACTERÍSTICAS CLAVE

SSG
PRE-RENDERING ESTÁTICO

PCI-DSS
PAGO DELEGADO STRIPE

RLS
ROW-LEVEL SECURITY ACTIVO

CSP
HEADERS DE SEGURIDAD

PRINCIPIOS RECTORES

Privilegio mínimo. Cada componente accede sólo a lo que necesita. La clave anónima del cliente no puede leer pedidos. Las credenciales potentes viven exclusivamente en funciones serverless.

Defensa en profundidad. Múltiples capas (CDN, CSP, RLS, JWT, validación servidor, captcha futuro) impiden que el fallo de una sola comprometa el sistema.

Estático por defecto. Lo que se puede pre-renderizar, se pre-renderiza. Reduce superficie de ataque y coste a la vez.

Auditable. Todo el código está versionado en Git. Cada despliegue es trazable. Las funciones serverless registran logs estructurados.

- **Resultado de la auditoría interna (mayo 2026):** sin vulnerabilidades críticas. Mejoras menores de SEO y endurecimiento de CSP en cola para próxima iteración. Detalle en sección 06.

02 Stack tecnológico

Componentes y proveedores en producción

FRONTEND Y COMPILACIÓN

FRAMEWORK

Astro 5

Generador estático con islas de hidratación. Build out: HTML + assets sin runtime servidor.

UI ISLAS

Vue 3

Composition API. Hidrata sólo el panel admin y el botón de compra; resto inerte.

ESTILOS

Tailwind CSS 3

Utility-first. Sin CSS custom salvo animaciones globales del layout.

BACKEND, DATOS Y SERVICIOS

BASE DE DATOS

Supabase / Postgres 17

PostgreSQL gestionado con RLS, vistas y triggers. Backups automáticos diarios.

AUTH

Supabase Auth

Magic-link por email para acceso al panel. Sin contraseñas que perder.

STORAGE

Supabase Storage

Bucket público `products`. Sólo admin escribe, lectura CDN. 10 MB/archivo, MIME whitelist.

FUNCIONES SERVERLESS

Supabase Edge Functions

Deno. 7 funciones desplegadas: pago, webhook, contacto, sync, rebuild, verify y email envíos.

PASARELA DE PAGO

Stripe Checkout

PCI-DSS Level 1 delegado. Webhooks firmados con HMAC-SHA256.

EMAIL TRANSACCIONAL

Resend

SPF/DKIM/DMARC alineados sobre dominio madcry.com. Plantillas HTML del estudio.

INFRAESTRUCTURA Y DESPLIEGUE

CDN / HOSTING

Firebase Hosting

Multi-región, HTTPS forzado, HTTP/2, rollback instantáneo a versión previa.

CI / CD

GitHub Actions

Workflow `deploy.yml` dispara build + deploy en cada push a `main` o vía panel admin.

DNS

A definir

Registro `madcry.com` apuntado a Firebase. Subdominio `www` redirige a apex.

> ¡ Todas las dependencias están **fijadas por versión** en `package.json` y `package-lock.json`. Ninguna es de terceros desconocidos: todas tienen mantenimiento activo y alta adopción comunitaria. La actualización de dependencias se ejecuta de forma controlada cada trimestre.

03 Arquitectura

Visión por capas y separación de responsabilidades

La plataforma sigue una arquitectura **JAMstack con backend gestionado**: el cliente es estático, los servicios sensibles viven detrás de funciones serverless aisladas, y los proveedores externos (Stripe, Resend) se invocan exclusivamente desde el lado servidor con credenciales nunca expuestas al navegador.

DIAGRAMA DE CAPAS

CAPA 1 · CLIENTE
Navegador
- Astro HTML estático
- Vue 3 (islas)
- Sólo claves públicas
- CSP estricta + HSTS

CAPA 2 · BORDE
CDN + Funciones
- Firebase Hosting
- Edge Functions (Deno)
- JWT + CORS allowlist
- Rate-limit por IP

CAPA 3 · NÚCLEO
Datos y pago
- Postgres con RLS
- Stripe (PCI-DSS L1)
- Resend (email)
- Storage + bucket auth

▲ ▼ comunicación cifrada TLS 1.3

RELACIÓN ENTRE COMPONENTES

```
┌──────────────┐    ┌──────────────────┐    ┌──────────────────┐
│  Visitante   │──▶│ Firebase CDN     │──▶│ HTML estático    │
│  navegador   │    │ (HTTPS+HSTS)     │    │ de /dist/        │
└──────────────┘    └──────────────────┘    └──────────────────┘
        │
        │ click "comprar" / submit form
        ▼
┌──────────────────────────────────────────────────────────────┐
│  Supabase Edge Functions (Deno · TLS · CORS allowlist)       │
│  — create-checkout-session  — verify-session                 │
│  — stripe-webhook (HMAC)    — send-contact-email             │
│  — admin-product-sync (JWT) — trigger-rebuild (JWT)          │
└──────────────────────────────────────────────────────────────┘
        │                        │                        │
        ▼                        ▼                        ▼
┌──────────────┐          ┌──────────────┐          ┌──────────────┐
│ Postgres     │          │ Stripe       │          │ Resend       │
│ + RLS        │          │ Checkout     │          │ SMTP API     │
│ + Storage    │          │ + webhooks   │          │ SPF/DKIM     │
└──────────────┘          └──────────────┘          └──────────────┘
```

▣ **Lectura clave:** el navegador *nunca* habla directamente con Stripe ni con Postgres con privilegios elevados. Toda operación sensible pasa por una Edge Function que valida origen, autenticación y permisos antes de tocar el dato.

04 Modelo de datos

Esquema PostgreSQL · públicas y privadas

Cuatro tablas centrales bajo el esquema `public`. Todas tienen **RLS activado** (Row-Level Security) — el motor de Postgres aplica las políticas a cada consulta, independientemente del cliente que la haga.

TABLA	DATOS SENSIBLES	POLÍTICA RLS APLICADA
`products` Catálogo de obra	Slug, nombre, descripción, precio, imágenes, IDs Stripe.	**Lectura pública** sólo de productos no-borrador. **Escritura admin** (email `info@madcry.com`).
`orders` Pedidos	Email cliente, nombre, teléfono, dirección de envío, importe, IDs Stripe.	**Sin acceso público.** Sólo admin lee/escribe; webhooks usan rol service_role que ignora RLS.
`messages` Contacto	Nombre, email, mensaje, IP, user-agent.	**Inserción pública** (form contacto). **Lectura sólo admin.**
`categories` Colecciones	Slug, nombre, label de precio, flag `hide_price`.	**Lectura pública.** Escritura admin.

VISTAS Y DERIVADOS

`products_public` · vista que filtra `status <> 'draft'` y une categoría. Es la única superficie expuesta al cliente para listados; los productos en borrador nunca son visibles aunque alguien manipule el cliente.

ALMACENAMIENTO DE ARCHIVOS

Bucket `products` (Supabase Storage), público en lectura, escritura limitada a admin. Tipos MIME permitidos: `image/jpeg, image/png, image/webp, image/avif`. Tamaño máximo por archivo: **10 MB**. Cabeceras CDN `Cache-Control: 31536000` para inmutabilidad por path.

DATOS PERSONALES TRATADOS

CATEGORÍA	ORIGEN	BASE LEGAL	PLAZO
Datos de pedido (nombre, email, dirección, teléfono)	Stripe Checkout	Ejecución de contrato	5 años (obligación fiscal)
Mensajes de contacto	Formulario público	Interés legítimo / consentimiento	2 años o hasta retirada

CATEGORÍA	ORIGEN	BASE LEGAL	PLAZO
Email del administrador	Supabase Auth	Ejecución del servicio	Mientras dure la relación

05 Flujos críticos

Compra de obra, formulario de contacto, publicación admin

FLUJO 1 · COMPRA DE UNA PIEZA

```
[1]  Cliente entra en /product/<slug> (HTML estático servido por CDN)
[2]  Click en "Adquirir Obra"
[3]  Navegador llama a create-checkout-session (Edge Function)
       → CORS verificado contra allowlist
       → Lee precio FRESCO de la BD (no confía en el cliente)
       → Verifica status='available' y categoría no es por encargo
[4]  Crea Stripe Checkout Session con price_data inline
[5]  Pre-crea fila en orders con status='pending'
[6]  Devuelve URL de Stripe; navegador redirige al cliente
[7]  Cliente paga en dominio Stripe (PCI-DSS)
[8]  Stripe llama al webhook stripe-webhook
       → Firma HMAC verificada con STRIPE_WEBHOOK_SECRET
       → Marca pedido como 'paid', producto como 'sold'
       → Envía email cliente (Resend) y email admin
[9]  Cliente vuelve a /success?session_id=cs_...
[10] Página llama a verify-session
       → Confirma con Stripe que el cobro está pagado
       → Devuelve email enmascarado (no completo)
```

✓ **Garantía técnica:** el precio nunca puede ser manipulado desde el navegador. La Edge Function relee el valor de Postgres en cada checkout. Si un atacante intentara inyectar otro `price_cents`, el servidor lo descartaría.

FLUJO 2 · MENSAJE DE CONTACTO / COMISIÓN

```
[1]  Visitante envía formulario en /contact
[2]  Edge Function send-contact-email ejecuta:
       → Honeypot anti-bots (campo invisible)
       → Validación email + límite de 5000 caracteres
       → Rate-limit: 3 mensajes/hora por email · 10/hora por IP
       → Inserta en tabla messages
       → Notifica a info@madcry.com vía Resend
[3]  Confirmación al usuario (200 OK con CORS allowlist)
```

FLUJO 3 · PUBLICACIÓN DE UNA OBRA DESDE EL PANEL ADMIN

```
[1]  Susana abre /admin y solicita magic-link a su email
[2]  Supabase Auth envía email con token de un solo uso
[3]  Click en el link → sesión JWT en el navegador
[4]  Carga del panel Vue (AdminPanel · island hidratada)
[5]  Susana edita / crea producto, sube imagen al bucket
[6]  Click en "Guardar" → POST con JWT a admin-product-sync
       → requireAdmin() verifica JWT y email exacto
       → sanitizeImages() filtra javascript:, data:, vbscript:
       → Crea/actualiza producto en BD
       → Sincroniza nombre / imágenes con Stripe Product
[7]  Click en "Publicar al sitio" → trigger-rebuild
       → Throttle de 60 s (anti-bucle)
       → Dispara workflow GitHub Actions
       → Build Astro + deploy Firebase (~2-3 min)
[8]  Sitio público actualizado · siguiente visitante ve la nueva obra
```

Inventario de funciones serverless

Las 7 Edge Functions desplegadas y su propósito

FUNCIÓN	AUTH	ORIGEN	PROPÓSITO
`create-checkout-session`	Pública	Cliente web	Crea sesión Stripe con precio fresco de BD.
`stripe-webhook`	Firma HMAC	Stripe	Marca pedido pagado, manda emails, archiva refunds.
`verify-session`	Pública	Cliente web	Verifica con Stripe el estado de pago de un session_id.
`send-contact-email`	Pública + rate-limit	Cliente web	Persiste mensaje y envía notificación al admin.
`admin-product-sync`	JWT admin	Panel Vue	CRUD de productos con sincronización a Stripe.
`send-shipping-email`	JWT admin	Panel Vue	Notifica al cliente con número de seguimiento.
`trigger-rebuild`	JWT admin	Panel Vue	Lanza build de Astro vía GitHub Actions.

06 Seguridad — capas y controles

Defensa en profundidad sobre la plataforma

Los controles se agrupan en cuatro **capas concéntricas**: red, transporte, aplicación y datos. Un fallo en una capa no compromete las siguientes.

CAPA DE RED Y TRANSPORTE

01	**HTTPS FORZADO**	Todo el tráfico circula sobre TLS 1.3. HTTP redirige a HTTPS por defecto en Firebase Hosting.
02	**HSTS PRELOAD**	`Strict-Transport-Security: max-age=31536000; includeSubDomains; preload` — el navegador rechaza HTTP futuro durante 1 año.
03	**CDN DISTRIBUIDA**	Mitigación implícita de DDoS volumétrico vía red global de Firebase (Cloud CDN de Google).

CAPA DE APLICACIÓN · CABECERAS

04	**CONTENT-SECURITY-POLICY**	Política estricta: `default-src 'self'`; conexiones limitadas a Supabase/Stripe; `frame-ancestors 'none'` impide clickjacking.
05	**X-FRAME-OPTIONS DENY**	Refuerzo doble anti-clickjacking para navegadores antiguos.
06	**X-CONTENT-TYPE-OPTIONS**	`nosniff` — el navegador no interpreta MIME types incorrectos.
07	**REFERRER-POLICY**	`strict-origin-when-cross-origin` — no fuga de paths privados a terceros.
08	**PERMISSIONS-POLICY**	Bloquea cámara, micrófono, geolocalización; permite `payment` sólo a dominios de Stripe.
09	**CROSS-ORIGIN-OPENER-POLICY**	`same-origin` — aísla la pestaña principal de pop-ups maliciosos.

CAPA DE APLICACIÓN · API

10	**CORS ALLOWLIST**	Las funciones serverless sólo aceptan peticiones de `madcry.com` y entornos de desarrollo. Antes era abierto a todos los dominios.
11	**VERIFICACIÓN JWT**	Funciones admin (`admin-product-sync`, `trigger-rebuild`, `send-shipping-email`) exigen JWT válido + email exacto del admin.
12	**VERIFICACIÓN DE FIRMA STRIPE**	El webhook valida HMAC-SHA256 con `STRIPE_WEBHOOK_SECRET`. Sin firma válida no se procesa el evento.

13	**RATE-LIMIT POR IP Y EMAIL**	Formulario de contacto: máx. 3/h por email + 10/h por IP. Función rebuild: throttle 60 s.
14	**HONEYPOT ANTI-BOTS**	Campo invisible en el formulario. Bots lo rellenan, humanos no. Reduce ~95% del spam automatizado.
15	**SANITIZACIÓN DE URLS DE IMAGEN**	Antes de tocar Stripe / BD, se filtran esquemas `javascript:`, `data:`, `vbscript:`.

CAPA DE DATOS

16	**ROW-LEVEL SECURITY**	Postgres aplica políticas a cada fila. La clave anon del cliente **no puede** leer pedidos ni mensajes ni borradores. Verificable por SQL.
17	**VISTA PÚBLICA LIMITADA**	`products_public` excluye productos en borrador. Es la única superficie expuesta en build.
18	**AISLAMIENTO DE SERVICE_ROLE**	La clave maestra que ignora RLS vive sólo dentro de Edge Functions, nunca en el navegador, nunca en logs.
19	**ALMACENAMIENTO CON MIME WHITELIST**	Bucket `products` sólo acepta JPEG, PNG, WebP, AVIF. Tope de 10 MB por archivo.
20	**CIFRADO EN REPOSO**	Todos los datos en Postgres y Storage cifrados con AES-256 por la propia plataforma de Supabase.
21	**EMAIL ENMASCARADO EN RESPUESTAS PÚBLICAS**	`verify-session` devuelve sólo iniciales + dominio (`su***@dominio.com`) para evitar enumerar clientes con un session_id filtrado.
22	**SOFT-DELETE DE PRODUCTOS**	No se borran filas: pasan a status `draft` y el producto Stripe se archiva. Mantiene integridad referencial con pedidos históricos.

PAGOS · SEPARACIÓN DE RESPONSABILIDADES

▣ Madcry Studio **nunca** almacena ni recibe datos de tarjeta. El proceso de cobro ocurre íntegramente en infraestructura de Stripe (PCI-DSS Level 1, máximo nivel de cumplimiento). Madcry sólo recibe del webhook el resultado del pago, los datos de envío del cliente y el identificador opaco `cs_…` de la sesión.

RESUMEN — AUDITORÍA INTERNA 2026-05

ÁREA	ESTADO	NOTAS
Secretos en repositorio	OK	Verificado: ningún `.env` en historial Git.
RLS en todas las tablas sensibles	OK	4/4 tablas con RLS activo y políticas explícitas.
Webhook Stripe firmado	OK	HMAC-SHA256 con secret rotable.
CORS de Edge Functions	OK	Allowlist tras endurecimiento de mayo 2026.
CSP sin `unsafe-inline`	MEJORA	Pendiente migración a hashes/nonces. No bloqueante.

ÁREA	ESTADO	NOTAS
Captcha en formularios	MEJORA	Honeypot + rate-limit cubren el riesgo actual. Cloudflare Turnstile en cola.
Validación de propiedad en /success	MEJORA	Email enmascarado mitigado; refuerzo opcional con cookie firmada.

07 Gestión de secretos y credenciales

Inventario, almacenamiento y procedimiento de rotación

Las credenciales se clasifican en dos grupos según su exposición:

PÚBLICAS (VISIBLES EN BUNDLE DEL NAVEGADOR)

VARIABLE	ORIGEN	RIESGO
`PUBLIC_SUPABASE_URL`	Supabase	Nulo · es URL pública del proyecto.
`PUBLIC_SUPABASE_ANON_KEY`	Supabase	Bajo · sólo permite lo que las políticas RLS permitan.
`PUBLIC_STRIPE_PUBLISHABLE_KEY`	Stripe	Nulo · diseñada para ir en cliente.

PRIVADAS (SOLO EN EDGE FUNCTIONS / GITHUB SECRETS)

VARIABLE	ALMACENAMIENTO	PROCEDIMIENTO DE ROTACIÓN
`SUPABASE_SERVICE_ROLE_KEY`	Edge Functions secrets	Dashboard Supabase → Settings → API → Reset.
`STRIPE_SECRET_KEY`	Edge Functions secrets	Stripe Dashboard → Developers → API Keys → Roll.
`STRIPE_WEBHOOK_SECRET`	Edge Functions secrets	Stripe Dashboard → Webhooks → Endpoint → Roll secret.
`RESEND_API_KEY`	Edge Functions secrets	Resend Dashboard → API Keys → Revoke + recreate.
`SUPABASE_DB_PASSWORD`	Solo terminal puntual	Supabase Dashboard → Database → Reset password.
`GITHUB_TOKEN` (deploy)	Edge Functions secrets	GitHub Settings → Developer settings → Tokens.
`FIREBASE_SERVICE_ACCOUNT`	GitHub Actions secrets	Firebase IAM → service account → New key.

REGLAS INTERNAS DE MANEJO

- [ ] Nunca pegar una credencial privada en chat, email, ticket o screenshot.
- [ ] Nunca persistirla en `.env.local` salvo si es estrictamente `PUBLIC_*` .

- [ ] Si es necesaria de forma puntual, exportarla en la sesión de terminal y borrar histórico (`history -c`).
- [ ] Rotar inmediatamente cualquier credencial que haya estado en un dispositivo perdido, repositorio público o canal compartido.
- [ ] Toda rotación queda registrada con fecha en este documento (sección Anexo · Registro de rotaciones).

> ! **Política de mínimo necesario:** el archivo `.env.local` que vive en la máquina de desarrollo contiene exclusivamente claves `PUBLIC_*`. Las claves potentes están en Supabase secrets — accesibles a la persona con login del proyecto, no al sistema de archivos del developer.

08 SEO, rendimiento y accesibilidad

Indexación y experiencia de usuario

SEO

CONTROL	ESTADO
Sitemap automático (`sitemap-index.xml`)	**ACTIVO** · 30 URLs · regenerado en cada build.
`robots.txt` con directivas explícitas	**OK** · admin/success bloqueados, IA crawlers permitidos.
Datos estructurados JSON-LD	**OK** · Organization + WebSite + Product + Article + Breadcrumb + Event.
Meta description única por página	**OK** · 11 páginas con descripción específica.
`<link rel="canonical">` en todas las páginas	**OK**
Open Graph + Twitter Card 1200×630	**OK** · `/images/og-banner.png`
`noindex` en /admin, /success, /404	**OK**

RENDIMIENTO

SSG (pre-render): el HTML llega listo desde la CDN, sin esperar a que un servidor lo construya.

Caché agresiva: assets versionados con `max-age=31536000, immutable`.

Lazy-loading en imágenes de listado.

Preload + onload-swap para Google Fonts (no bloquea el render inicial).

HTTP/2 y **Brotli** servidos por Firebase.

ACCESIBILIDAD

El sitio sigue las pautas **WCAG 2.1 AA** en navegabilidad (contrastes verificados, estructura semántica con `<main>`, `<nav>`, `<article>`, focus visibles, atributos `alt` en imágenes, formulario etiquetado). Auditoría de profundidad pendiente con herramienta automatizada (axe-core) prevista para próxima iteración.

09 Cumplimiento legal

RGPD, LSSI-CE y normativa aplicable

REGLAMENTO GENERAL DE PROTECCIÓN DE DATOS (RGPD · UE 2016/679)

REQUISITO	IMPLEMENTACIÓN
Información clara al usuario	Página `/privacidad` y `/legal` publicadas y enlazadas en footer.
Base legal documentada	Ejecución de contrato (pedidos), interés legítimo (contacto), consentimiento (newsletter futura).
Minimización de datos	Sólo se piden los datos imprescindibles para envío y facturación.
Derechos ARSULIPO	Solicitud por email a `info@madcry.com`. Acceso, rectificación, supresión, limitación, portabilidad y oposición.
Cifrado en tránsito y en reposo	TLS 1.3 + AES-256 por proveedor (Supabase, Stripe, Resend).
Encargados de tratamiento	Supabase, Stripe, Resend, Firebase. Todos con DPA firmado y cláusulas SCC para transferencia internacional.
Plazos de conservación	5 años para datos fiscales (pedidos), 2 años para mensajes de contacto.
Brechas de seguridad	Procedimiento documentado · notificación a la AEPD en plazo < 72 h.

LEY DE SERVICIOS DE LA SOCIEDAD DE LA INFORMACIÓN (LSSI-CE)

Aviso legal accesible desde el footer con titularidad, NIF, dirección y email de contacto.

Política de cookies (en cola — el sitio no instala cookies de terceros analíticos sin consentimiento).

Identificación clara del precio, IVA, gastos de envío y forma de pago antes de la compra.

Confirmación de pedido por email con resumen y referencia.

COMERCIO ELECTRÓNICO

La tienda cumple con los **14 días de derecho de desistimiento** establecidos por la Ley General para la Defensa de Consumidores y Usuarios, salvo en piezas personalizadas (comisión a medida) donde el desistimiento queda excluido por su naturaleza específica conforme al art. 103.c) del TRLGDCU. Esta excepción se informa de forma destacada antes de finalizar el encargo.

10 Operación, backups y continuidad

Cómo se despliega, se respalda y se recupera la plataforma

DESPLIEGUE

Cada commit en la rama `main` dispara el workflow `deploy.yml` de GitHub Actions, que ejecuta `npm ci`, `astro build` y un deploy a Firebase Hosting con la action oficial. Tiempo total medio: ~2-3 minutos. Es posible disparar el mismo workflow manualmente desde el panel admin (función `trigger-rebuild`).

BACKUPS

COMPONENTE	FRECUENCIA	RETENCIÓN	MECANISMO
PostgreSQL	Diaria automática	7 días (plan free) 30 días (plan Pro)	Supabase Backups (PITR opcional)
Imágenes (Storage)	Continuo	Indefinido	Replicación interna del proveedor
Código	En cada commit	Indefinido	GitHub (clones distribuidos)
Versión desplegada	En cada build	10 versiones	Firebase Hosting · rollback en 1 click

RECUPERACIÓN ANTE INCIDENCIAS

ESCENARIO	PROCEDIMIENTO	RTO OBJETIVO
Bug en producción	Rollback en Firebase a versión anterior	< 5 min
Pérdida de datos en BD	Restauración de backup Supabase + replay de webhooks	< 4 h
Compromiso de credencial	Rotación según matriz de la sección 07 + auditoría de logs	< 1 h
Caída del proveedor	Comunicación al cliente · esperar SLA del proveedor	Según SLA proveedor

MONITORIZACIÓN

Logs estructurados en cada Edge Function (`console.log` visible en Supabase Dashboard). Stripe envía email automático ante webhooks fallidos. Resend reporta entrega/rebote. Próxima iteración: integración con **Sentry** para errores de cliente y agregación de logs en una sola consola.

11 Anexos

Glosario, matriz de roles y registro

A · GLOSARIO RÁPIDO

SSG	Static Site Generation. El sitio se compila a HTML estático en build, no en cada petición.
RLS	Row-Level Security. Postgres aplica políticas por fila a cualquier consulta.
JWT	JSON Web Token. Identidad firmada que el navegador presenta para autenticarse.
HMAC	Algoritmo que firma un mensaje con un secreto compartido para verificar su origen.
CSP	Content-Security-Policy. Cabecera HTTP que limita qué recursos puede cargar el navegador.
HSTS	HTTP Strict Transport Security. Fuerza HTTPS futuras visitas.
PCI-DSS	Payment Card Industry Data Security Standard. Lo cumple Stripe en nombre de Madcry.
JAMSTACK	JavaScript, APIs y Markup pre-renderizado. Arquitectura donde el cliente es estático y los servicios se invocan vía API.

B · MATRIZ DE ROLES

ROL	QUIÉN	ACCESO
Visitante anónimo	Cualquiera	Lectura del catálogo público, blog, compras, formulario contacto.
Cliente con pedido	Comprador	Página /success de su pedido, emails de confirmación y envío.
Administradora	Susana Madriz (info@madcry.com)	Panel /admin: CRUD productos, mensajes recibidos, lanzar publicaciones, enviar shipping.
Mantenimiento técnico	Tane Solutions	Repositorio Git, Supabase Dashboard, Stripe Dashboard, Firebase Console (acceso revocable).

C · REGISTRO DE AUDITORÍAS Y ROTACIONES

FECHA	EVENTO	RESPONSABLE
2026-05-04	Auditoría interna de seguridad y SEO. Endurecimiento de CORS, sanitización de imágenes, throttle en rebuild, sitemap automático, JSON-LD, descripciones únicas. Sin hallazgos críticos.	Tane Solutions
2026-05-04	Limpieza de `.env.local` · retiradas service_role, stripe_secret, webhook_secret, resend_key, db_password.	Tane Solutions
—	*Próxima rotación trimestral programada*	*Tane Solutions*

▣ **Documento controlado.** Este documento se versiona junto al código en `docs/`. Cualquier cambio en la arquitectura, dependencia o control de seguridad debe reflejarse aquí en la misma pull request.

Plataforma construida con criterio.

Cualquier consulta sobre arquitectura, seguridad o cumplimiento se responde por escrito por la entidad responsable del mantenimiento. Datos de contacto al pie.